

Eliminación de carreras críticas mediante semaforos

Nuria Guerra Casal Adrian Gijon Yubero
Grao en Enxeñaría Informática

1. Introducción

En el apartado 1 se demostró que el acceso concurrente al buffer sin sincronización produce carreras críticas: el valor de `cima` se corrompe, el consumidor retira guiones en lugar de datos reales y los conteos de vocales divergen. Este apartado implementa la solución clásica al problema productor-consumidor utilizando **semaforos POSIX con nombre**, que bloquean a cada proceso exactamente cuando no puede continuar (buffer lleno o vacío) y garantizan que solo uno de los dos procesos accede al buffer en cada momento.

Los semaforos POSIX con nombre (`sem_open()`) son adecuados aquí porque los dos procesos son independientes (no se trata de hilos) y pueden ser abiertos por cualquier proceso que conozca su nombre, sin necesidad de parentesco entre ellos.

2. Mecanismo de sincronización: los tres semaforos

La solución utiliza tres semaforos, cada uno con un rol distinto:

- **VACIAS** (valor inicial $N = 10$): cuenta los huecos libres en el buffer. El productor hace `sem_wait(vacias)` antes de insertar, bloqueándose si el buffer está lleno, y el consumidor hace `sem_post(vacias)` al retirar un elemento, incrementando el contador.
- **CHEAS** (valor inicial 0): cuenta los elementos disponibles para consumir. El consumidor hace `sem_wait(cheas)` antes de retirar, bloqueándose si el buffer está vacío, y el productor hace `sem_post(cheas)` al insertar.
- **MUTEX** (valor inicial 1): semáforo binario que garantiza exclusión mutua en el acceso al buffer. Sólo uno de los dos procesos puede estar dentro de la región crítica en cada momento.

El protocolo completo de inserción, que elimina por completo la carrera crítica del apartado 1, es el siguiente:

```
1 sem_wait(vacias); //espera hueco libre (bloquea si buffer esta lleno)
2 sem_wait(mutex); //entra en exclusion mutua
3
4 //Region critica, acceso seguro al buffer
5 datos->pila[datos->cima] = caracter;
6 datos->cima++;
7
8 sem_post(mutex); //sale de la region critica
9 sem_post(cheas); //avisa al consumidor de que hay un elemento nuevo
```

Listing 1: Protocolo de inserción con semaforos en `insert_item()`

El consumidor sigue el protocolo simétrico: `sem_wait(cheas)`, `sem_wait(mutex)`, acceso al buffer, `sem_post(mutex)`, `sem_post(vacias)`. Gracias al mutex, los dos pasos que antes estaban separados por un `sleep()` (al leer `cima` y escribir en el buffer) ahora son atómicos desde el punto de vista del otro proceso.

3. Gestión de los semaforos

El productor es responsable de **crear e inicializar** los semaforos con `sem_open(..., O_CREAT, 0700, valor)`. Como buena práctica, los elimina primero con `sem_unlink()` por si quedaron activos de una ejecución anterior que no hubiera terminado limpiamente:

```
1 sem_unlink("/VACIAS");
2 sem_unlink("/CHEAS");
3 sem_unlink("/MUTEX");
4
5 sem_t *vacias = sem_open("/VACIAS", O_CREAT, 0700, TAMANO_BUFFER);
6 sem_t *cheas = sem_open("/CHEAS", O_CREAT, 0700, 0);
7 sem_t *mutex = sem_open("/MUTEX", O_CREAT, 0700, 1);
```

Listing 2: Creación de semaforos en `prod_sem.c`

El consumidor **abre** los semaforos ya existentes sin reinicializarlos, usando `sem_open(..., 0)`. Si el consumidor intentara crear los semaforos con sus propios valores iniciales, sobrescribiría el estado actual y rompería la sincronización. Al terminar, el productor llama a `sem_close()` y `sem_unlink()` para cerrarlos y eliminarlos del sistema. Mientras, el consumidor solo hace `sem_close()`.

4. Velocidades y fases

Para poder observar el comportamiento del buffer en situaciones extremas (lleno y vacío), ambos procesos adaptan su velocidad en tres fases mediante `sleep()`s colocados **fuera de la región crítica**:

Iteraciones	Productor	Consumidor
0 – 29	Sin sleep (rapido)	Sleep 3s (lento) → buffer se llena
30 – 59	Sleep 3s (lento)	Sin sleep (rapido) → buffer se vacia
60 – 79	Sleep aleatorio 0–3s	Sleep aleatorio 0–3s

Es importante que los `sleep()` estén fuera de la región crítica, ya que dentro de ella bloquean el mutex e impiden que el otro proceso avance.

5. Ejecución

```
# Compilar ambos programas
gcc -o prod_sem prod_sem.c
gcc -o cons_sem cons_sem.c

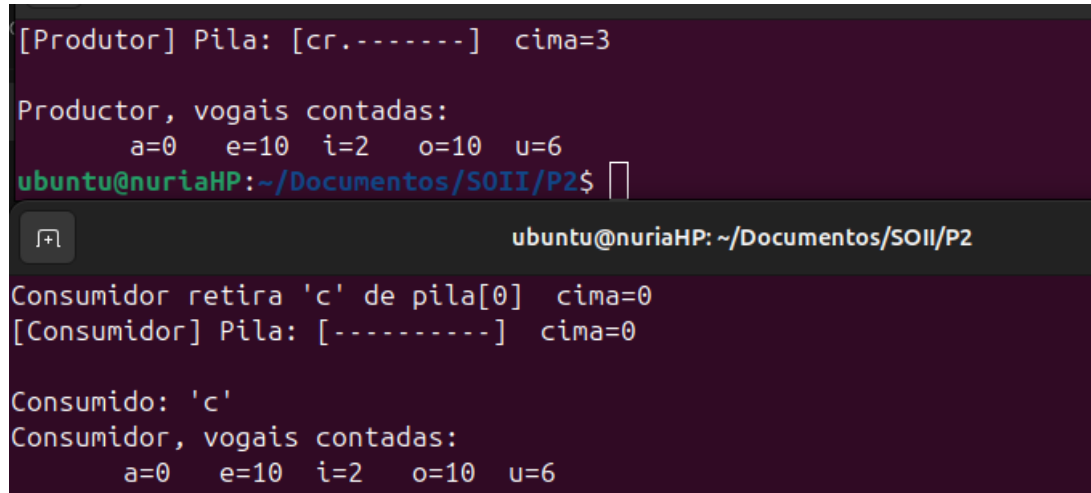
# Terminal 1: lanzar el productor primero (crea la memoria compartida y los
#             semaforos)
./prod_sem texto.txt

# Terminal 2: lanzar el consumidor despues
./cons_sem
```

El productor debe lanzarse antes que el consumidor, ya que crea el fichero de memoria compartida y los semaforos. El consumidor espera automáticamente hasta que el fichero `buffer.dat` existe antes de intentar abrirlo. Ambos procesos realizan exactamente 80 iteraciones y terminan solos, sin necesidad de interrumpirlos manualmente. Al finalizar, cada proceso imprime su conteo de vocales.

6. Resultados y conclusiones

La diferencia más evidente respecto al apartado 1 se observa en el **conteo final de vocales**: con semaforos, los totales del productor y del consumidor coinciden, lo que confirma que ningún carácter se ha perdido ni duplicado. El consumidor nunca retira guiones '-' en lugar de datos reales, y el valor de **cima** evoluciona de forma monótona y predecible, sin saltos anómalos.



```
[Produtor] Pila: [cr.-----] cima=3

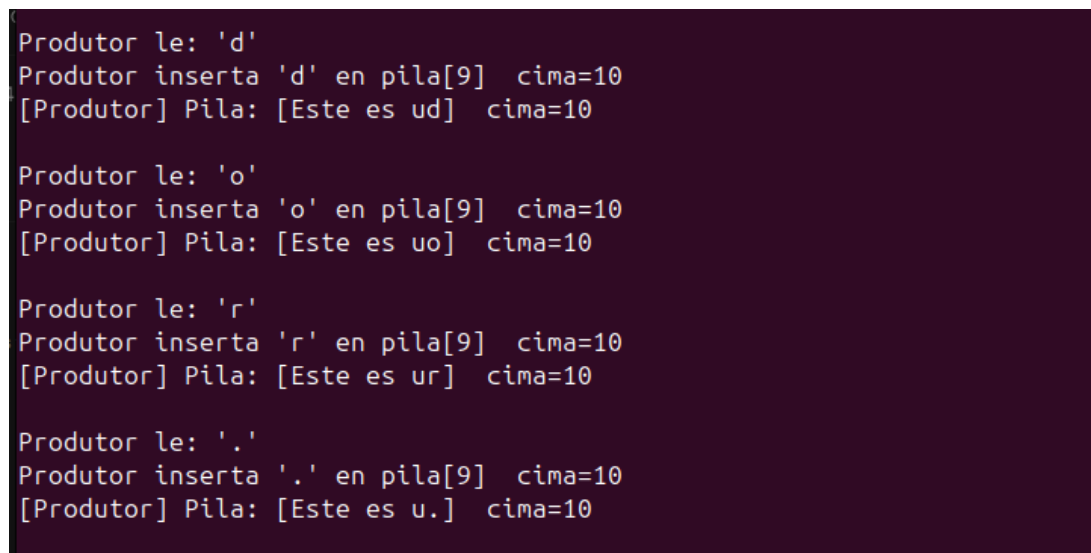
Produtor, vogais contadas:
    a=0   e=10  i=2   o=10  u=6
ubuntu@nuriaHP:~/Documentos/SOII/P2$

ubuntu@nuriaHP: ~/Documentos/SOII/P2
Consumidor retira 'c' de pila[0] cima=0
[Consumidor] Pila: [-----] cima=0

Consumido: 'c'
Consumidor, vogais contadas:
    a=0   e=10  i=2   o=10  u=6
```

Figura 1: Captura de conteo final de vocales.

Durante la primera fase (iteraciones 0-29) el buffer se llena rápidamente hasta **cima=10** y el productor queda bloqueado en **sem_wait(vacias)**, la salida deja de mostrar mensajes del productor hasta que el consumidor retira un elemento cada 3 segundos. En la segunda fase (iteraciones 30-59) ocurre lo contrario, el consumidor vacía el buffer hasta **cima=0** y queda bloqueado en **sem_wait(cheas)** entre cada producción del productor.



```
Produtor le: 'd'
Produtor inserta 'd' en pila[9] cima=10
[Produtor] Pila: [Este es ud] cima=10

Produtor le: 'o'
Produtor inserta 'o' en pila[9] cima=10
[Produtor] Pila: [Este es uo] cima=10

Produtor le: 'r'
Produtor inserta 'r' en pila[9] cima=10
[Produtor] Pila: [Este es ur] cima=10

Produtor le: '.'
Produtor inserta '.' en pila[9] cima=10
[Produtor] Pila: [Este es u.] cima=10
```

Figura 2: Captura de bloqueo del productor al no poder insertar en el buffer.

La conclusion de este apartado es que los semaforos resuelven completamente el problema de las carreras críticas observado en el apartado 1. El semáforo **vacias** reemplaza la espera activa del productor cuando el buffer esta lleno; **cheas** reemplaza la espera activa del consumidor cuando esta vacío; y **mutex** hace que la región crítica sea atómica desde el punto de vista del proceso contrario. El resultado es un programa correcto, predecible y sin pérdida de datos.